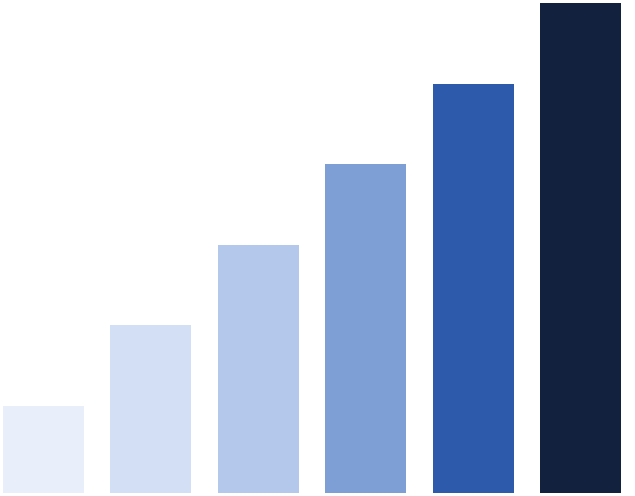
EMBEDDED SOFTWARE PORTFOLIO

매일 조금씩 더 나아지는 개발자 이상민입니다.



Embedded Software Engineer · 2026



이상민

임베디드 소프트웨어 · 신입

이메일: abc101357@gmail.com

연락처: 010-9658-8515

Github: https://github.com/namesangmin

블로그: https://lee-jobs.tistory.com

학력

고등학교: 성남고등학교 (2016.3 ~ 2019.2)

대학교: 강원대학교 AI소프트웨어학과 (2020.3 ~ 2026.2)

자격증

- 정보처리기사 (2025.12)
- 리눅스마스터 (2026.4)
- OPIc Intermediate High (2026.2)

Programming

C / C++ / Python

Embedded / Platform

STM32 / Raspberry Pi / FreeRTOS / Linux

Development Tools

Git / VS Code / STM32CubeMX / Linux perf / libCacheSim

PROJECTS

레이더 표적 탐지 시스템

GitHub ↗ P.2 – P.6

2026.04 – 2026.06 | 팀 6인, 기여도 40% | LIG D&A The Ssen 임베디드 SW 스쿨

레이더 모의신호를 실시간으로 수신, 처리해 표적을 탐지하고 결과를 운용통제 PC에 전시하는 시스템

MY ROLE

4-Core 파이프라인 설계, 동기화와 분배 방식 실측 선택, 메모리 접근 최적화

KEY RESULT

요구 주기 51.2ms 충족

48.4 → 16.4ms

UGV 로봇 원격 조종 시스템

GitHub ↗ P.7 – P.8

2026.01 – 2026.03 | 팀 5인, 기여도 30% | LIG D&A The Ssen 임베디드 SW 스쿨

조종부의 입력을 받아 UGV의 주행, 회전, 로봇팔을 제어하는 시스템

MY ROLE

조작부 하드웨어 구성, ADC 입력 처리, 모드 전환 구조 설계

KEY RESULT

하나의 조종부로 3가지 조작

LIRS 캐시 교체 알고리즘 연구

P.9 – P.10

2024.10 – 2025.03 | 학부연구생 연구 | 강원대학교 SSRLab

캐시 크기에 따른 LIRS 알고리즘의 미스율 변화를 분석한 연구

MY ROLE

LIRS 알고리즘 분석, 공개 트레이스 분석, 검증용 트레이스 설계

KEY RESULT

미스율 상승 원인 확인

스택 크기 비율 조정으로 미스율 17.9%p 감소

PROJECT 01

레이더 표적 탐지 시스템

2026.04 – 2026.06 | 팀 6인, 기여도 40% | LIG D&A The Ssen 임베디드 SW 스쿨

레이더 모의신호를 실시간으로 수신, 처리해 표적을 탐지하고 결과를 운용통제 PC에 전시하는 시스템

목표 **51.2ms마다 들어오는 레이더 데이터를, 다음 데이터가 오기 전에 처리**

0.1ms 간격으로 펄스가 512개 들어와 51.2ms마다 한 묶음이 됨(512 x 1001 I/Q 데이터)

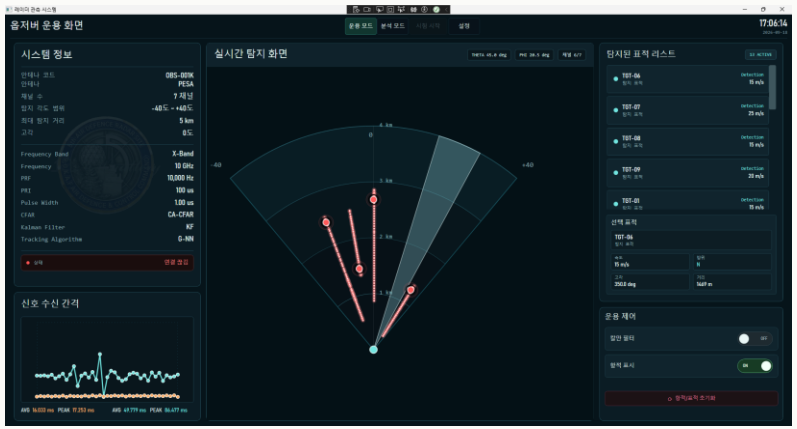


① 신호모의기

② 신호처리기 **담당 영역**

- Raspberry Pi 5의 4-Core 구조와 동기화 설계
- 병목 분석 및 성능 최적화

③ 운용통제 PC



신호처리 결과를 거리, 속도 기반 표적으로 탐지하여 실시간 전시

문제

실시간 처리 불가

- 초기 구현 처리 시간 약 **150ms**, 주기의 3배
- ArmPL 적용 후 **48.4ms**, 전송할 시간 부족

→

접근

성능 분석을 바탕으로 단계적 구조 개선

- 단계별 처리 시간을 측정하고 perf로 병목 분석
- 병렬화 → 분배와 동기화 → 메모리 접근 → 수신과 처리 중첩 (P.3~P.6)

→

결과

실시간 연속 처리 달성

48.4ms → 16.4ms

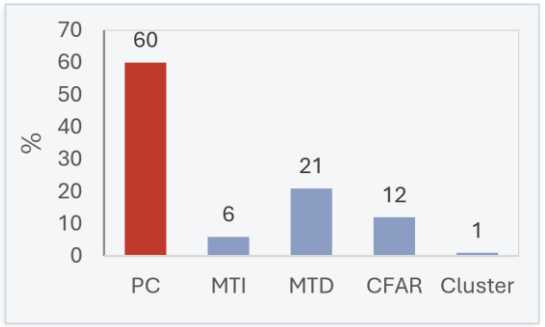
PROJECT 01

4-Core 신호처리 파이프라인 설계

처리 시간의 60%를 차지한 Pulse Compression을 2개 Worker로 나눠 병렬 처리

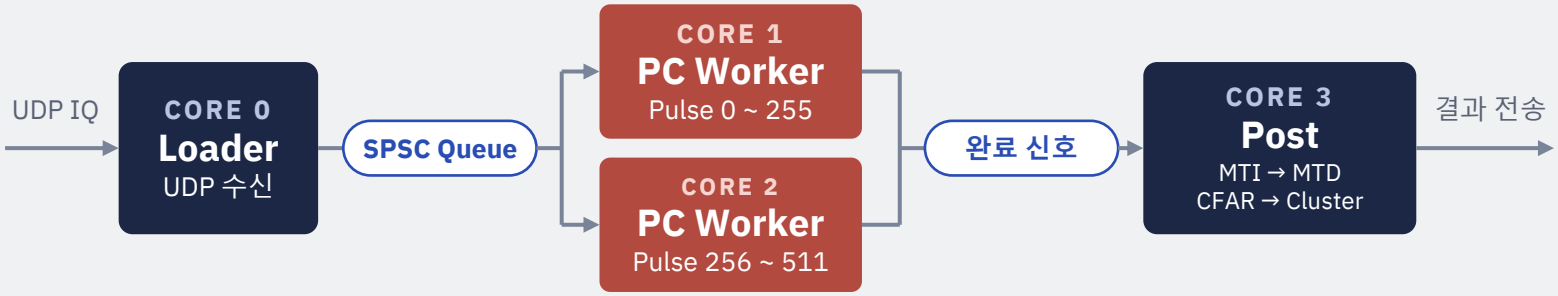
분석 결과

▶ 단계별 처리 시간 비율(1 코어)



Pulse Compression이 전체의 60% 차지

파이프라인 구조



설계 판단

가장 무거운 단계만 2개 Worker로 분할 처리

- 각 펄스의 Range 방향 Pulse Compression은 서로 독립적으로 처리 가능
- 펄스 0~255 / 256~511을 Worker 2개가 나눠서 처리

설계 검증

PC Worker 1개 → 2개

20.9ms → 16.4ms

다른 최적화는 같은 조건, Worker 수만 변경

PROJECT 01

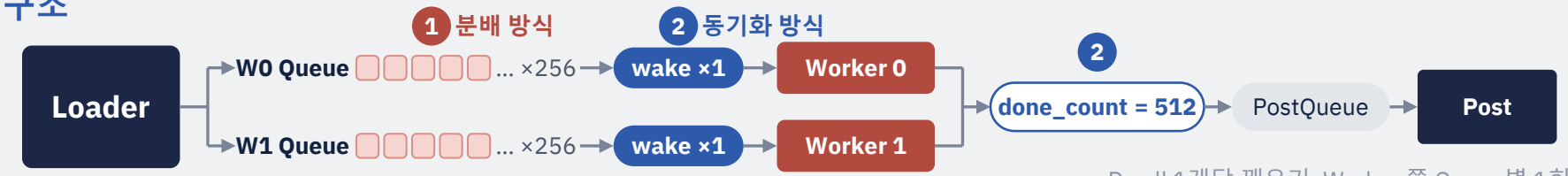
코어 간 작업 전달과 동기화

작업 분배 방식과 Worker 동기화 방식을 각각 측정하여 결정

실험 방법 공통 인터페이스 + Factory로 구현만 교체해, 같은 입력과 같은 파이프라인에서 A/B 비교

perf stat | perf trace | clock_gettime()

최종 구조



Dwell 1개당 깨우기: Worker 쪽 Queue별 1회 + Post 쪽 마지막 Worker가 1회

1 분배 방식

Even/Odd → Half (연속 블록)

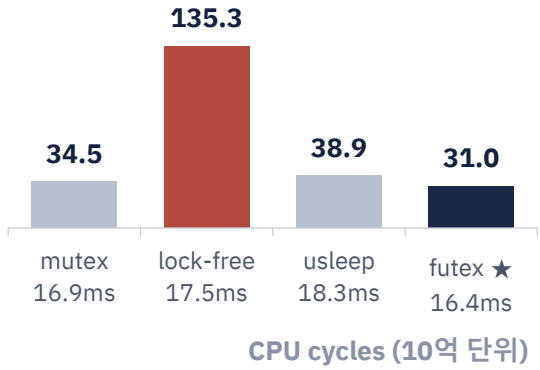


- Even/Odd: 4MB 전체를 16KB stride로 접근
- Half: Worker마다 연속된 2MB 블록 접근 → prefetch에 유리

16.99ms → 16.4ms | IPC 1.28 → 1.35

2 동기화 방식

CPU 비용을 비교해 futex 선택



- 선택: lock-free는 대기 중 busy-wait로 futex 대비 cycles 4.4배
- 구현: Worker별 Queue에 256개 펄스 번호를 넣고 1회 wake

16.4ms | 31.0B cycles

PROJECT 01

캐시에 맞춘 메모리 접근

신호처리 단계별 접근 방향에 맞춰 메모리 배치와 전치 방식을 변경

데이터 구조
1 Dwell = 펄스 512 × 샘플 1001 (I/Q, 약 4MB)

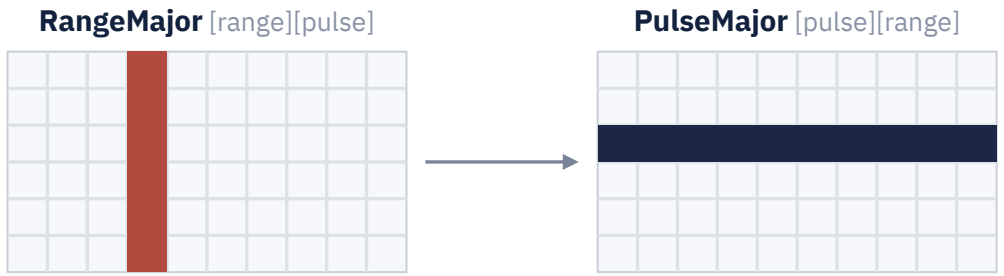
Pulse Compression (처리 시간의 60%)
펄스마다 **샘플 방향**으로 읽음

MTI / MTD
샘플 위치마다 **펄스 방향**으로 읽음

1 메모리 배치

RangeMajor → PulseMajor

- 초기: MTI/MTD가 읽는 **펄스 방향**에 맞춰 저장
- 측정 결과, PC가 60% → PC가 읽는 **샘플 방향**으로 저장 순서 변경

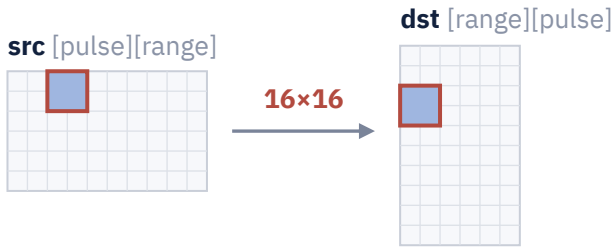


53.7ms → 16.4ms | IPC 0.41 → 1.35

동일 조건에서 Layout만 변경

2 Tiled Transpose

ArmPL 대신 Tiled Transpose를 직접 구현, 타일을 L1 캐시에 맞춤

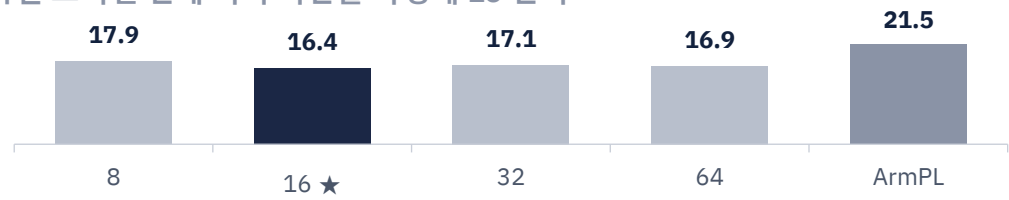


Cortex-A76 L1 64KB

16×16 타일 = 4KB

(src 2KB + dst 2KB)

타일 크기별 전체 처리 시간을 측정해 16 선택



ArmPL 21.5ms → 16.4ms

동일 조건에서 Transpose만 변경

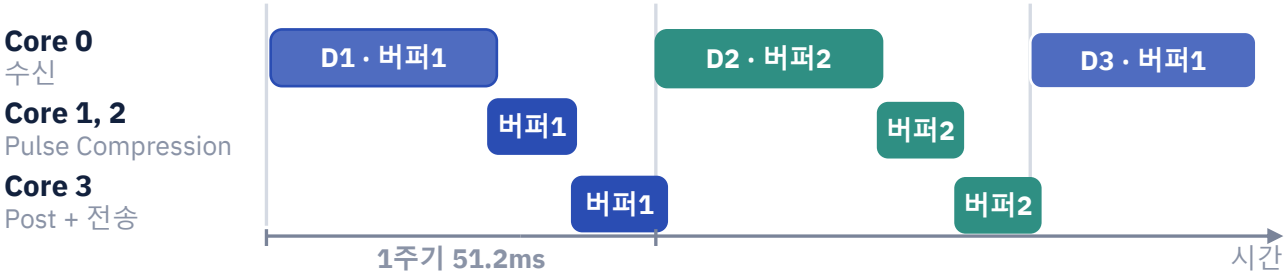
PROJECT 01

실시간 연속 처리 완성

처리 시간을 줄이고 수신과 처리를 겹쳐, 51.2ms 주기 안에서 연속 처리

Double Buffering 기반 파이프라인 동작

파이프라인 단계를 코어별로 나눠 동시에 진행



버퍼 교대 Core 0이 두 버퍼를 번갈아 채워 처리 중인 데이터의 덮어쓰기 방지

처리 중첩 이전 Dwell 처리가 끝나기 전에 다음 Dwell 수신 시작

최종 성과

처리 시간

48.4ms → 16.4ms

P.3~P.5 구조 설계와 최적화

요구 주기 51.2ms 충족

탐지 오차: 거리 0.40%, 속도 0.45%

마무리

기여	4-Core 파이프라인 설계, 동기화와 분배 방식 실측 선택, 메모리 접근 최적화
익힌 것	**멀티코어 스레드 설계** | **측정 기반 성능 최적화** | **팩토리 패턴 기반 구현 교체 구조** | **ICD 기반 UDP 통신** C, Linux, pthread, futex, ArmPL, perf
아쉬운 점	결과 전송(약 12.85ms)이 Post 코어에 몰림 → Core 0이나 별도 전송 스레드로 분리했다면 Post 부담을 더 줄일 수 있었을 것

PROJECT 02

UGV 로봇 원격 조종 시스템

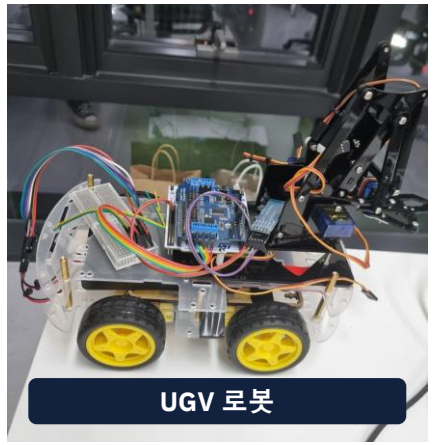
2026.01 – 2026.03 | 팀 5인, 기여도 30% | LIG D&A The Ssen 임베디드 SW 스쿨

조종부의 입력을 받아 UGV의 주행, 회전, 로봇팔을 제어하는 시스템

목표 하나의 조종부에서 주행, 회전, 작업 모드를 전환하며 조작



담당 영역 · Controller



UGV 로봇

모드별 입력

같은 컨트롤러라도 모드에 따라 쓰는 입력이 다름

모드	사용하는 입력	제어 대상
주행	조이스틱 X/Y	→ 차량 주행
회전	조이스틱 + 회전 버튼	→ 제자리 회전
작업	가변저항 ×3 + 그리퍼 버튼	→ 로봇팔, 그리퍼

문제

모드마다 사용하는 입력과 해석 방식이 다름

- 주행, 회전, 작업마다 필요한 입력 조합이 다름
- 조작하지 않아도 입력값이 미세하게 변동

→

접근

모드별 입력 해석 분리

- 입력은 공통 수집하고 현재 모드에서만 해석
- 의미 없는 작은 변화는 제어값에서 제외

→

결과

하나의 조종부로 3가지 조작

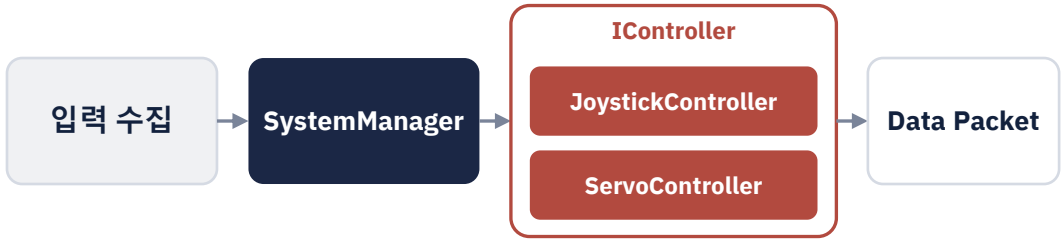
- 모드 전환만으로 주행, 회전, 작업 제어
- 미세한 입력 변화가 제어값에 반영되지 않음

PROJECT 02

모드 전환 구조와 입력 변화 필터링

모드별 입력 해석을 분리하고, 의미 없는 작은 변화를 제어값에서 제외

1 모드별 Controller 분리 Strategy Pattern



입력 모드와 상관없이 항상 수집

전환 모드 변경 시 활성 Controller만 교체

3가지 조작을 하나의 구조로 처리

2 입력값 흔들림 억제



문제 조작하지 않아도 ADC 값이 조금씩 변함 (UART 로그로 발견)

해결 12-bit ADC 범위의 5%를 기준으로, 204 이하 변화는 미반영

미세한 떨림이 제어값에 반영되지 않음

마무리

기여	조작부 하드웨어 구성, ADC 입력 처리, 모드 전환 구조 설계
익힌 것	**전략 패턴과 다형성 이해** | **UART 로그 기반 디버깅** | **ADC 입력 처리** STM32CubeMX, VS Code, FreeRTOS, C++
아쉬운 점	입력값 흔들림을 줄였지만, 적용 전후 개선 정도를 수치로 비교하지 못함

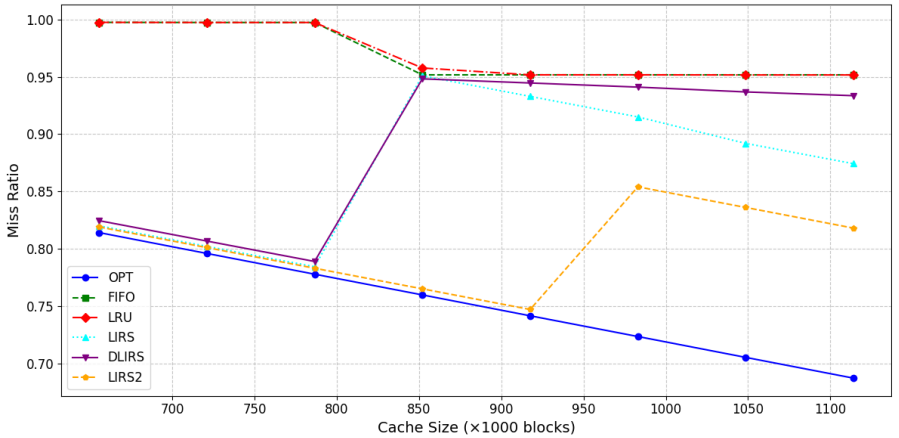
PROJECT 03

LIRS 캐시 교체 알고리즘 연구

2024.10 – 2025.03 | 학부연구생 연구 | 강원대학교 SSRLab | 공개 트레이스 약 3,000개

캐시 크기에 따른 LIRS 알고리즘의 미스율 변화를 분석한 연구

목표 **실제 스토리지 사용 환경에서 캐시 용량을 늘렸는데 미스율이 오르는 원인을 분석하고 개선 방향 탐색**



LIRS 동작 방식

Hot (LIR)	자주 쓰는 데이터, 캐시에 유지
Cold (HIR)	덜 쓰는 데이터, 먼저 내보냄
승격	Cold 데이터가 다시 쓰이면 Hot으로 이동

문제

캐시를 늘렸는데 미스율 상승

- LIRS 계열에서만 최대 약 **16%p** 상승
- DLIRS, LIRS2에서도 같은 현상 확인

→

접근

요청 순서별 데이터 이동 추적

- 미스율이 회복되지 않는 구간 집중 분석
- 해당 구간에서 반복되는 패턴을 비교해 원인 후보를 좁힘

→

결과

원인 확인, 미스율 17.9%p 감소

- 직접 설계한 트레이스로 같은 현상 재현
- 스택 크기 비율 조정으로 이상 현상 완화

PROJECT 03

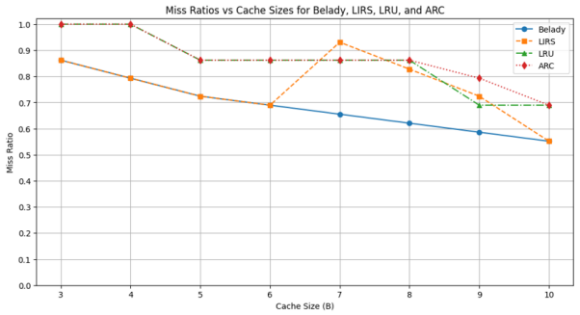
캐시 증가 시 미스율 상승 원인 분석

직접 설계한 트레이스로 현상을 재현하고, 조건을 하나씩 바꿔가며 원인을 검증

접근 — **재현** 논문의 현상을 직접 재현 → **탐색** 회복 실패 구간 분석 → **확인** 요청 몰림(burst) 확인 → **가설** Hot 승격 방식 의심

원인 — 다시 쓰이지 않을 데이터가 Hot으로 승격 → 기존 Hot 데이터가 밀려남 → **캐시 오염, 미스율 상승**

1 직접 설계한 트레이스로 재현

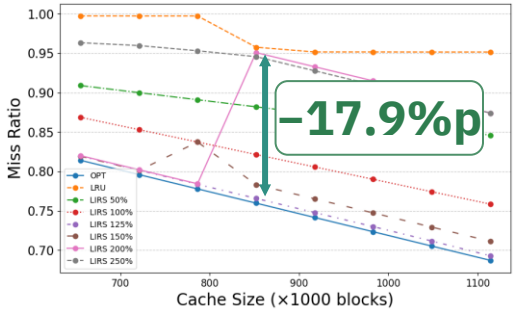


캐시 6 → 7블록, LIRS 미스율

0.69 → 0.93

승격 후 다시 참조되지 않는 요청 순서를 직접 설계

2 조건을 바꿔 원인 확인



스택 크기 비율에 따른 미스율

125% 이하 이상 현상 없음

150% 소폭 상승

200% 이상 LRU 수준까지 저하

원인으로 본 조건만 고정하자 이상 현상이 사라짐

마무리

기여	LIRS 알고리즘 분석, 공개 트레이스 분석, 검증용 트레이스 설계
익힌 것	**캐시 교체 알고리즘 이해** | **변수 통제 실험** | **트레이스 기반 원인 분석** libCacheSim, Python(matplotlib)
아쉬운 점	원인과 해결 방향은 찾았지만, 캐시 크기에 맞춰 자동으로 조절하는 알고리즘까지는 만들지 못함